

```
-- inbuilt functions:-
-- Inbuilt functions in MySQL are predefined functions provided by MySQL to perform
-- specific operations on data
-- They help you with
-- 1) Perform calculations
-- 2) Manipulate strings
-- 3) Work with dates
-- 4) Handle null values
-- 5) summarize data
-- 6) Convert data types
```

```
-- 1) Aggregate functions:-
-- 2) Numeric functions:-
-- 3) String functions:-
-- 4) Date and time functions:-
-- 5) Null functions :-
-- 6) Conditional functions:-
```

```
create database InBuiltFunctions;
use InBuiltFunctions;
create table employees(e_id int primary key auto_increment,
e_name varchar(225) not null,
e_salary decimal(9,2)) auto_increment =100;
```

```
insert into employees (e_name, e_salary)
values("Ganesh",54000),
("Priya",null);
```

```
select * from employees;
```

```
select max(e_salary) from employees;
```

```
-- aliases
```

```
-- aliases is used to change column and table name to some other name temporarily
select max(e_salary) as "Highest_salary" from employees;
```

```
-- 1) Aggregate functions:-
-- Aggregate functions perform calculations on multiple rows and return a single
-- value.
-- They summarize data.
```

```
-- there are 5 primary aggregate functions:_
-- 1) count(), 2) sum(), 3) avg(), 4) min(), 5) max()
select * from employees;
```

```
-- count total number of employees in our company
```

```
-- 1) count()
```

```
-- count is used to count the number of rows
```

```
-- count(*) :- counts all the rows including the null values
```

```
select count(*) as total_employees from employees;
```

```
-- count(column_name) :- count only non-null values in that column
select count(e_id) as total_employees from employees;
```

```
alter table employees add column phone_number varchar(225) unique;
select count(phone_number) as "total employees with a phone number"from employees;
```

```
-- 2) Sum():-
-- sum is used to add numeric values together.
select * from employees;
select sum(e_salary) from employees;
```

```
-- 3) avg():-
select avg(coalesce(e_salary,0)) from employees;
select avg(e_salary) from employees;
```

```
select sum(e_salary) from employees;
select count(*) from employees;
```

```
select * from employees;
```

```
-- 3) avg():-
-- finds average value
select avg(coalesce(e_salary,0)) from employees;
select avg(e_salary) from employees;
```

```
-- 4) min():-
-- returns smallest value
select min(e_salary) from employees;
```

```
-- 5) max():-
-- returns highest value
select max(e_salary) from employees;
```

```
create table employee_departments
(emp_id int primary key auto_increment,
emp_name varchar(225) not null,
department_name varchar(225),
location varchar(225),
salary decimal(9,2)) auto_increment = 101;
```

```
insert into employee_departments (emp_name, department_name,
location, salary)
values
("Sahil", "HR", "Pune", 45000),
("Sanjay","IT","Mumbai",50000),
("Sonali","HR","Banglore",90000),
("Ravi","IT","Pune",78000),
```

```
("Rohit","Cloud","Pune",34000),
("Rani","Cloud","Banglore",56000),
("Vaishali","HR","Mumbai",60000),
("Vinay","IT","Pune",99000),
("Varun","Cloud","Mumbai",67000);
```

```
select * from employee_departments;
-- find the highest salary in emp_dept table
select max(salary) from employee_departments;
-- find the highest salary in each city
update employee_departments set salary = null where emp_id = 103;
```

```
-- Group by:-
-- group by is used to group up similar values together. It's mainly use with
aggregate functions.
-- The group by clause is used to group rows that have the same values in specified
columns into summary rows.
-- It is almost always used with aggregate functions to perform calculations on
each group, rather than on the
-- entire table.
```

```
-- syntax:-
/*
select column_name1, column_name2, aggregate_functions(column_name)
from table_name
where condition
group by column_name1, column_name2
order by column_name1 asc/desc
limit 1 offset 1;
*/
```

```
select location, max(salary)
from employee_departments
group by location;
```

```
-- banglore 90000
-- mumbai   67000
-- pune     99000
```

```
-- find the highest salary in each department
select department_name, max(salary)
from employee_departments
group by department_name;
```

```
-- find the average salary of each department
select department_name, avg(coalesce(salary,0))
from employee_departments
```

```

group by department_name;

-- order by:-
-- order by is used to rearrange our records in ascending or descending order
according
-- to a column.
-- By default order by is in ascending order so asc is optional
-- desc :- desc is used for descending order. --> order by column_name desc;
-- asc :- asc is used for ascending order. --> order by column_name asc;

-- example :- order employees from highest to lowest earner
select *
from employee_departments
order by salary desc;

-- find the avg salary of each location and return the result from highest average
earning
-- location to lowest average earning location
select * from employee_departments;

select location, avg(salary) as avg_salary
from employee_departments
group by location
order by avg_salary desc;

-- find the lowest average salaried location from your table.

-- -----
-- limit and offset

-- find the lowest earning employee from your table.
select *
from employees
order by e_salary asc
limit 1;

-- find the second lowest earning employee from your table
select *
from employees
order by e_salary asc
limit 1 offset 1;

-- find the third and fourth highest earning employees from your tables
select *
from employees
order by e_salary desc
limit 2 offset 2;

select *
from employees

```

```
order by e_salary desc
limit 4,1;
```

```
-- having clause:-
select * from employee_departments;
```

```
select location, avg(salary) as avg_salary
from employee_departments
group by location
having avg_salary > 58000;
```

```
-- find all the departments with more than 2 employees
select * from employee_departments;
```

```
select department_name, count(department_name) as total_employees
from employee_departments
group by department_name
having total_employees > 2;
```

```
-- order by :- Order by is clause is used to sort the result set of a query in
-- ascending or descending order based on one or more columns.
-- key points:-
-- default sorting is ascending(asc)
-- used desc for descending order
-- can sort by multiple columns
-- executed after select but before limit
```

```
-- syntax :-
select column1, column2
from table_name
order by column1 asc|desc;
```

```
-- limit clause :-
-- the limit clause is used to restrict the number of rows
-- returned by a query.
-- key points :-
-- mostly used with order by
-- unefull for pagination
-- MySQL specific(not standard SQL)
```

```
-- offset clause:-
-- the offset clause is used to skip a specific number
-- of rows before starting to return rows.alter
-- used with limit
-- important for pagination
```

-- skips rows

-- having clause:-

-- The having clause is used to filter grouped records after applying group by

-- where filters rows before grouping

-- having filters groups after grouping

-- key points :- used with group by

-- works with aggregate functions like

-- count(), sum(), max(), min(), avg()

-- Execute order

-- 1) From

-- 2) where

-- 3) group by

-- 4) having

-- 5) select

-- 6) order by

-- 7) limit

```
-- inbuilt functions:-
-- Inbuilt functions in MySQL are predefined functions provided by MySQL to perform
-- specific operations on data
-- They help you with
-- 1) Perform calculations
-- 2) Manipulate strings
-- 3) Work with dates
-- 4) Handle null values
-- 5) summarize data
-- 6) Convert data types
```

```
-- 1) Aggregate functions:-
-- 2) Numeric functions:-
-- 3) String functions:-
-- 4) Date and time functions:-
-- 5) Null functions :-
-- 6) Conditional functions:-
```

```
create database InBuiltFunctions;
use InBuiltFunctions;
create table employees(e_id int primary key auto_increment,
e_name varchar(225) not null,
e_salary decimal(9,2)) auto_increment =100;
```

```
insert into employees (e_name, e_salary)
values("Ganesh",54000),
("Priya",null);
```

```
select * from employees;
```

```

select max(e_salary) from employees;
-- aliases
-- aliases is used to change column and table name to some other name temporarily
select max(e_salary) as "Highest_salary" from employees;

-- 1) Aggregate functions:-
-- Aggregate functions perform calculations on multiple rows and return a single
-- value.
-- They summarize data.

-- there are 5 primary aggregate functions:_
-- 1) count(), 2) sum(), 3) avg(), 4) min(), 5) max()
select * from employees;

-- count total number of employees in our company
-- 1) count()
-- count is used to count the number of rows
-- count(*) :- counts all the rows including the null values
select count(*) as total_employees from employees;
-- count(column_name) :- count only non-null values in that column
select count(e_id) as total_employees from employees;

alter table employees add column phone_number varchar(225) unique;
select count(phone_number) as "total employees with a phone number" from employees;

-- 2) Sum():-
-- sum is used to add numeric values together.
select * from employees;
select sum(e_salary) from employees;

-- 3) avg():-
select avg(coalesce(e_salary,0)) from employees;
select avg(e_salary) from employees;

select sum(e_salary) from employees;
select count(*) from employees;

select * from employees;

-- 3) avg():-
-- finds average value
select avg(coalesce(e_salary,0)) from employees;
select avg(e_salary) from employees;

-- 4) min():-
-- returns smallest value
select min(e_salary) from employees;

```

```
-- 5) max():-
-- returns highest value
select max(e_salary) from employees;
```

```
create table employee_departments
(emp_id int primary key auto_increment,
emp_name varchar(225) not null,
department_name varchar(225),
location varchar(225),
salary decimal(9,2)) auto_increment = 101;
```

```
insert into employee_departments (emp_name, department_name,
location, salary)
values
("Sahil", "HR", "Pune", 45000),
("Sanjay", "IT", "Mumbai", 50000),
("Sonali", "HR", "Banglore", 90000),
("Ravi", "IT", "Pune", 78000),
("Rohit", "Cloud", "Pune", 34000),
("Rani", "Cloud", "Banglore", 56000),
("Vaishali", "HR", "Mumbai", 60000),
("Vinay", "IT", "Pune", 99000),
("Varun", "Cloud", "Mumbai", 67000);
```

```
select * from employee_departments;
-- find the highest salary in emp_dept table
select max(salary) from employee_departments;
-- find the highest salary in each city
update employee_departments set salary = null where emp_id = 103;
```

```
-- Group by:-
-- group by is used to group up similar values together. It's mainly use with
aggregate functions.
-- The group by clause is used to group rows that have the same values in specified
columns into summary rows.
-- It is almost always used with aggregate functions to perform calculations on
each group, rather than on the
-- entire table.
```

```
-- syntax:-
/*
select column_name1, column_name2, aggregate_functions(column_name)
from table_name
where condition
group by column_name1, column_name2
order by column_name1 asc/desc
limit 1 offset 1;
*/
```

```
select location, max(salary)
from employee_departments
group by location;
```

```
-- banglore 90000
-- mumbai   67000
-- pune     99000
```

```
-- find the highest salary in each department
select department_name, max(salary)
from employee_departments
group by department_name;
```

```
-- find the average salary of each department
select department_name, avg(coalesce(salary,0))
from employee_departments
group by department_name;
```

```
-- order by:-
-- order by is used to rearrange our records in ascending or descending order
according
-- to a column.
-- By default order by is in ascending order so asc is optional
-- desc :- desc is used for descending order. --> order by column_name desc;
-- asc :- asc is used for ascending order. --> order by column_name asc;
```

```
-- example :- order employees from highest to lowest earner
select *
from employee_departments
order by salary desc;
```

```
-- find the avg salary of each location and return the result from highest average
earning
-- location to lowest average earning location
select * from employee_departments;
```

```
select location, avg(salary) as avg_salary
from employee_departments
group by location
order by avg_salary desc;
```

```
-- find the lowest average salaryed location from your table.
```

```
-- -----
-- limit and offset
```

```
-- find the lowest earning employee from your table.
```

```

select *
from employees
order by e_salary asc
limit 1;

-- find the second lowest earning employee from your table
select *
from employees
order by e_salary asc
limit 1 offset 1;

-- find the third and fourth highest earning employees from your tables
select *
from employees
order by e_salary desc
limit 2 offset 2;

select *
from employees
order by e_salary desc
limit 4,1;

-- having clause:-
select * from employee_departments;

select location, avg(salary) as avg_salary
from employee_departments
group by location
having avg_salary > 58000;

-- find all the departments with more than 2 employees
select * from employee_departments;

select department_name, count(department_name) as total_employees
from employee_departments
group by department_name
having total_employees > 2;

-- order by :- Order by is clause is used to sort the result set of a query in
-- ascending or descending order based on one or more columns.
-- key points:-
-- default sorting is ascending(asc)
-- used desc for descending order
-- can sort by multiple columns
-- executed after select but before limit

-- syntax :-

```

```
select column1, column2
from table_name
order by column1 asc|desc;
```

```
-- limit clause :-
-- the limit clause is used to restrict the number of rows
-- returned by a query.
-- key points :-
-- mostly used with order by
-- unefull for pagination
-- MySQL specific(not standard SQL)
```

```
-- offset clause:-
-- the offset clause is used to skip a specific number
-- of rows before starting to return rows.alter
-- used with limit
-- important for pagination
-- skips rows
```

```
-- having clause:-
-- The having clause is used to filter grouped records after applying group by
-- where filters rows before grouping
-- having filters groups after grouping

-- key points :- used with group by
-- works with aggregate functions like
-- count(), sum(), max(), min(), avg()
```

```
-- -----
```

```
-- Execute order
-- 1) From
-- 2) where
-- 3) group by
-- 4) having
-- 5) select
-- 6) order by
-- 7) limit
```

